

Opus Publica

Policy-as-Code Developer Guide

Writing Rego Policies for the Continuous Certification Engine

Version 1.0

Issued 09 August 2026

Document Type	Technical Documentation — Developer Guide
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Engineers writing Rego policies for constitutional provisions; CI/CD engineers integrating policy gates
Builds On	RC2 Evolution Roadmap Ch 4 (Continuous Certification Engine); RC1 Engineering Constitution Vol I + Vol II; RC1 Certification Checklist (600+ criteria)
Classification	Mandatory — RC2 Implementation Specification
Effective Date	09 August 2026
Review Cadence	Quarterly, or on any OPA / Confest / Gatekeeper major release
Document ID	OP-RC2-IS-003

The Constitution is not prose to be read. It is code to be executed. This Guide teaches the language in which the code is written.

Chapter 1 — The Constitution as Executable Code

1.1 Opening Hook: The Constitution as Executable Code

Every provision of the Opus Publica Engineering Constitution — the 38 engineering invariants, the 47 non-negotiable rules, the 276 canonical fields, the 34 events, the 10 SLOs, the 103 thresholds, the 600+ certification criteria — exists, today, as prose in PDF documents. The prose is precise. The prose is comprehensive. The prose has been reviewed, signed off, and issued under the authority of the Office of the Chief Systems Architect. The prose is also, until it is encoded, unenforceable by any agent faster or more reliable than a human reader. An engineer who introduces a pull request that violates INV-I-02 (exactly one DOI per published article version) does not, in the absence of executable policy, learn of the violation until a reviewer reading the Constitution notices it — or, more often, until the violation reaches production and corrupts the scholarly record. The Continuous Certification Engine (CCE) of RC2 closes that gap by encoding every provision as a Rego policy that executes in CI, on every pull request, before any human reviewer sees the change. This Guide is the document that teaches engineers how to write that code.

Rego is the language in which the Constitution's provisions are written as enforceable code. Rego is a declarative, Datalog-derived policy language defined by the Open Policy Agent (OPA) project, a CNCF Graduated policy engine. A Rego policy is not a program in the imperative sense: it does not execute a sequence of statements. It declares, instead, a set of facts and rules, and the OPA engine derives the consequences. The consequences, for Opus Publica's purposes, are the deny messages that a policy emits when a constitutional provision is violated. The engineer who writes Rego is the engineer who decides, in unambiguous code, what the Constitution means in every conceivable runtime state. This Guide teaches that decision.

The Guide's Prime Directive

Every constitutional provision that CAN be expressed as a Rego policy SHALL be expressed as a Rego policy. Every Rego policy SHALL be testable, and SHALL be tested, before it is merged. Every merged policy SHALL run in CI on every pull request whose change touches the provision the policy encodes. No provision SHALL be enforced by a policy whose test suite does not include at least one PASS case and at least one FAIL case. A policy without tests is not a policy; it is a draft.

1.2 Purpose, Audience, and Prerequisites

The purpose of this Guide is to teach engineers and CI/CD integrators how to write, test, deploy, and operate Rego policies that encode the provisions of the Opus Publica Engineering Constitution. The Guide is the developer-facing companion to RC2 Evolution Roadmap Chapter 4, which specifies the Continuous Certification Engine at the architectural level. Where the Roadmap says “each criterion

SHALL be encoded as a Rego policy module,” this Guide says “here is how to write that module, here is how to test it, here is how to wire it into CI, here is how to debug it when it produces a false positive.”

The Guide addresses two primary audiences:

- **Policy authors** — the engineer who reads a constitutional provision, derives the predicate it specifies, and writes the Rego policy that enforces the predicate; and
- **CI/CD integrators** — the engineer who wires the policy into a GitHub Actions workflow, chooses between Conftest and opa eval, configures report-only versus blocking mode, and emits the OSCAL Assessment Results fragment that becomes part of the release's certification record.

The prerequisites for using this Guide effectively are:

1. **Constitution familiarity (REQUIRED).** the reader has read at least the chapter of the Engineering Constitution that contains the provision they intend to encode, and can locate the provision by its stable ID (e.g., INV-I-02, PUB-001, ARCH-003);
2. **Rego familiarity (HELPFUL).** exposure to the OPA documentation at openpolicyagent.org, and ideally the completion of the OPA interactive tutorial; familiarity is helpful but not required because Chapter 3 of this Guide covers the fundamentals;
3. **Tooling (REQUIRED).** the reader has a working local installation of the OPA binary (version 0.60.0 or later) and the Conftest binary (version 0.45.0 or later), and has a clone of the Machine-Readable Constitution (MRC) repository with its policies/ and tests/ directories;
4. **RC2 Roadmap context (REQUIRED).** the reader has read the RC2 Evolution Roadmap Chapter 4 and understands the criterion-to-policy mapping (one criterion, one policy, one CI job).

1.3 Scope

This Guide covers the full lifecycle of a Rego policy in the Continuous Certification Engine: authoring from a constitutional provision, testing, review, merge, deployment to CI and to the Gatekeeper admission controller, and operational debugging. The Guide is bound by the Constitution: every policy this Guide teaches engineers to write encodes a provision the Constitution has already established. The Guide does not establish new provisions; it does not interpret the Constitution beyond what the Constitution's text and the Chief Systems Architect's precedents specify; and it does not authorize the engineer to enforce, by policy, any rule the Constitution does not state. A policy that exceeds the Constitution is a defect of the same severity as a policy that fails to enforce the Constitution.

The Guide is explicitly out of scope for the following:

- the choice of OPA over alternative policy engines (Cedar, OpenFGA); that choice is made in RC2 ADR-004 and is not re-litigated here;
- the governance of the Machine-Readable Constitution repository (branches, owners, review cadence); that is the subject of RC2 ADR-002;
- the design of the OSCAL Assessment Results schema; this Guide emits OSCAL-AR fragments but defers to NIST SP 800-204C and the OSCAL 1.1 specification for the schema itself;

- the operational dashboarding of policy results; that is the Governance Dashboard (RC2 Roadmap Chapter 5).

1.4 Document Conventions

This Guide uses the requirement vocabulary of IETF RFC 2119. The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” are used throughout with the precise meanings defined in RFC 2119. Code examples are presented in monospaced code blocks; each code block declares its language (rego, yaml, bash, json) on the first line as a comment or, for YAML, by the structure of the content. File paths are given relative to the root of the Machine-Readable Constitution repository (e.g., policies/INV-I-02.rego).

Convention	Example	Notes
Provision ID	INV-I-02	Stable identifier from the Constitution; never invented by the policy author.
Policy file	policies/INV-I-02.rego	One file per provision; file name = provision ID.
Test file	tests/INV-I-02_test.rego	One test file per policy; the _test suffix is mandatory for OPA's test runner.
Package	package opus_publica.invariants	Maps to the Constitution's provision class (invariants, rules, contracts, fields, events, slos, thresholds).
Deny message	INV-I-02 FAIL: ...	Every deny message SHALL start with the provision ID and the word FAIL.
CI workflow	.github/workflows/policy-test.yml	Lowercase, hyphenated; one workflow per policy cadence (PR, nightly, release).

1.5 References

The following external references are cited throughout this Guide. The engineer is expected to consult them for canonical detail; this Guide summarizes only what is needed to write Opus Publica policies.

Reference	Source	Authority
Open Policy Agent documentation	openpolicyagent.org/docs	CNCF Graduated project; canonical OPA reference.

Rego reference	openpolicyagent.org/docs/latest/policy-reference	The Rego language specification and built-in function catalog.
Conftest documentation	conftest.dev	OPA subproject; canonical Conftest reference.
OPA Gatekeeper	openpolicyagent.org/docs/latest/kubernetes	Canonical Gatekeeper (admission controller) reference.
NIST SP 800-204C	csrc.nist.gov/publications/detail/sp/800-204c/final	Implementation of DevSecOps for a Microservices-based Application with Service Mesh; policy-as-code guidance.
GitHub Actions documentation	docs.github.com/en/actions	Canonical reference for the CI/CD substrate.
OSCAL 1.1	pages.nist.gov/OSCAL	Open Security Controls Assessment Language; the Assessment Results layer is the bridge to external auditors.
RFC 2119	rfc-editor.org/rfc/rfc2119	Key words for use in RFCs to indicate requirement levels.

Chapter 2 — The Policy-as-Code Stack

2.1 Open Policy Agent (OPA)

Open Policy Agent is a general-purpose policy engine, originally created by Styra and donated to the CNCF in 2018, where it graduated in February 2021. OPA is a single binary that decouples policy decision-making from application code: the application asks OPA a question (“may this user perform this action?”, “does this configuration satisfy these constraints?”), OPA evaluates the relevant Rego modules against the input, and OPA returns a decision. The application does not implement policy; it queries OPA. The policy is not embedded in the application; it lives in version-controlled Rego files. This separation is the architectural foundation of policy-as-code: policy is authored, reviewed, tested, and deployed independently of the code it governs.

OPA exposes the decision API in three forms, each appropriate to a different integration point. The first is the in-process Go library, used when the application is itself a Go service and the latency of a localhost call is acceptable. The second is the REST API, served by the OPA daemon (`opa run --server`), used when the application is non-Go or when policies must be shared across services. The third is the CLI (`opa eval`), used by developers and by CI jobs to evaluate policies against input files. For Opus Publica, the CLI form is the primary integration in CI; the REST API form is used by the runtime sidecars that enforce policy in production; and the in-process Go library is used by the Conftest runner, which embeds OPA.

```
# bash — evaluating a policy locally with opa eval
opa                                     eval \
                                     --data policies/ \
                                     --input evidence/INV-I-02/sample-pass.json \
                                     --format pretty \
                                     'data.opus_publica.invariants.deny'

# expected output (PASS case): an empty array
# []
```

2.2 Rego

Rego is the policy language OPA evaluates. Rego is descended from Datalog, a declarative logic programming language with decades of academic and industrial heritage. The Datalog heritage manifests in three properties that distinguish Rego from imperative languages. First, Rego is declarative: a policy states what must be true, not how to compute it. The OPA engine derives the consequences; the policy author does not specify evaluation order. Second, Rego is closed-world: anything the policy does not derive as true is, by default, false. This is the source of the negation-as-failure semantics that engineers learning Rego frequently find surprising. Third, Rego is rule-based: a policy is a set of rules, each of which defines a relation. The deny/allow pattern that Opus Publica adopts is one instance of this rule-based structure.

The package/deny/allow pattern is the convention by which Opus Publica policies are structured. Every policy file declares a package (e.g., package opus_publica.invariants). Within the package, the policy defines one or more deny rules, each of which emits a human-readable message when the rule fires. The absence of any deny message is the policy's PASS; the presence of one or more deny messages is the policy's FAIL. The Conftest runner and the opa eval CLI both interpret an empty deny set as PASS and a non-empty deny set as FAIL. The allow rule, when present, is the dual of deny: it is true when no deny rule fires. Opus Publica policies use deny exclusively; allow is omitted unless the policy explicitly needs to assert a positive permission.

2.3 Conftest

Conftest is an OPA subproject, hosted at conftest.dev, that wraps OPA in a runner designed for testing structured configuration data. Where opa eval is a general evaluation tool, Conftest is purpose-built for the case “I have a YAML or JSON file and a Rego policy; tell me whether the policy passes or fails, and exit non-zero if it fails.” This is exactly the case that pre-merge CI gates need: a pull request introduces a YAML file (a contract, an evidence file, a Kubernetes manifest), and Conftest evaluates the relevant Rego policies against that YAML, exiting non-zero if any policy denies. Conftest integrates with OPA by embedding the OPA library; every Rego feature available in opa eval is available in Conftest.

```
#      bash      —      running      Conftest      against      a      YAML      file
conftest                                     test
                                     --policy                                     policies/
                                     evidence/INV-I-02/sample-pass.yaml

#      PASS      output      (exit      code      0):
#      FYI      —      1      file,      0      tests,      0      failures,      0      exceptions

#      FAIL      output      (exit      code      1):
#      FAIL      —      evidence/INV-I-02/sample-fail.yaml —      main —      INV-I-02      FAIL:
#      article      version      v3      has      2      DOIs      (expected      1)
#      1      test,      1      failure,      0      exceptions
```

Conftest is the integration point for Opus Publica's pre-merge CI: the GitHub Actions workflow [.github/workflows/conftest-verify.yml](https://github.com/opus-publica/opus-publica/blob/main/.github/workflows/conftest-verify.yml) invokes Conftest on every PR that touches the contracts/ directory, evaluating the relevant Rego policies against the changed YAML. Conftest is also the integration point for the policy author's local test loop: conftest test is the command an engineer runs, repeatedly, while developing a policy, before they ever push a commit.

2.4 GitHub Actions

GitHub Actions is the CI/CD substrate on which the Continuous Certification Engine runs. GitHub Actions is chosen because Opus Publica's source control is GitHub, because GitHub Actions integrates natively with pull requests (status checks, required checks, branch protection), and because GitHub Actions

runners can be ephemeral and OIDC-identified, which is the foundation of the supply-chain integrity controls (Sigstore signing, SLSA provenance) that RC2 introduces. The CCE defines four workflows, each with a different cadence and scope: policy-test (per PR touching policies/), conftest-verify (per PR touching contracts/), nightly-criterion (cron, all criteria), and release-certification (on release branch merge, all GATE criteria).

Each workflow runs the OPA or Conftest binary in a GitHub-hosted runner, evaluates the relevant policies against the relevant input, and produces a structured result. The structured result is an OSCAL Assessment Results fragment (Chapter 7 specifies the emission). The fragment is uploaded to the certification repository as evidence, and the workflow's status check reflects PASS or FAIL on the GitHub pull request UI. When the workflow is in blocking mode (Phase 3 of the RC2 migration), a FAIL status blocks the pull request from merging, enforced by GitHub's branch protection rule. When the workflow is in report-only mode (Phase 1), the FAIL status is visible but does not block; the parity dashboard tracks the rate of FAILs as the team prepares for the Phase 3 cutover.

2.5 OPA Gatekeeper

OPA Gatekeeper is the Kubernetes admission controller that enforces Rego policies at deploy time. Where Conftest enforces policies pre-merge (on the YAML a developer submits in a PR), Gatekeeper enforces policies pre-deploy (on the manifest the deployment pipeline submits to the Kubernetes API server). Gatekeeper is implemented as a Kubernetes admission webhook: every create, update, and delete operation against the cluster is intercepted by Gatekeeper, evaluated against the relevant Constraint Templates and Constraints (which embed Rego), and either admitted or rejected. A rejected admission returns the violation messages to the caller, which is the deployment pipeline; the pipeline fails fast, and the non-compliant deployment does not occur.

Gatekeeper is the deploy-time enforcement layer for the subset of Opus Publica policies that govern Kubernetes manifests: the policy that no deployment may proceed without an SLO defined, the policy that every deployment MUST have resource requests and limits, the policy that no image may be deployed without a Sigstore signature, and so on. Chapter 8 of this Guide specifies the mapping from Rego policies (authored as deny rules in policies/) to Gatekeeper Constraint Templates and Constraints, and the enforcement modes (dryrun, deny, warn) the team uses during the rollout of each new Gatekeeper-enforced policy.

2.6 The Policy Lifecycle

A Rego policy in Opus Publica moves through a six-stage lifecycle. Each stage is owned by a role, has a defined entrance criterion, and produces a defined artifact. The lifecycle is binding: a policy that skips a stage is not a policy, it is a draft.

Stage	Owner	Entrance Criterion	Artifact
1. Author	Policy author (engineer)	A constitutional provision	policies/<PROV-ID>.rego

		exists with a stable ID and a written predicate.	(draft)
2. Test	Policy author	Draft policy exists.	tests/<PROV-ID>_test.rego with ≥ 1 PASS and ≥ 1 FAIL case.
3. Review	Reviewer (a different engineer)	Draft policy + tests exist; local conf test passes.	Approved pull request.
4. Merge	Repository maintainer	Reviewer has approved; CI policy-test workflow is green.	Policy on the main branch.
5. Deploy	CI/CD engineer	Policy is merged; RC2 phase permits its activation.	Policy in the OPA bundle distributed to CI and sidecars; (for Gatekeeper policies) Constraint applied.
6. Monitor	On-call engineer	Policy is live.	Decision-log entries in the OPA decision log; parity dashboard entry; (on FAIL) an incident ticket.

The lifecycle is iterative: a policy that produces false positives in production (Chapter 9) returns to Stage 1 for revision; a policy that the Constitution amends returns to Stage 1 for re-authoring. The version of a policy is the git SHA of the commit at which it was merged; there is no separate version number, because the Constitution's provisions are versioned by amendment and the policy is bound to the provision, not to a release cadence of its own.

Chapter 3 — Rego Fundamentals for Opus Publica

3.1 The Rego File Structure

Every Rego file in Opus Publica has the same five-part structure: a package declaration, optional imports, metadata comments, the deny rules that constitute the policy, and optional helper rules. The package declaration **MUST** be the first non-comment line. The package name mirrors the Constitution's provision class: `opus_publica.invariants` for INV-* provisions, `opus_publica.rules` for NN-* provisions, `opus_publica.contracts` for SUB-* provisions, and so on. Imports pull in either built-in functionality or data from the Machine-Readable Constitution (e.g., the provision's formal predicate). The deny rules are the policy's enforcement: each deny rule fires when its body is satisfied, emitting a message that names the provision and explains the failure.

```
# policies/INV-I-02.rego
# Encoding of constitutional provision INV-I-02.
# Source: Vol II Ch 26 §26.2 (Engineering Invariants).
# PASS: COUNT(distinct dois WHERE article_version_id = V) = 1 for every published V.

package opus_publica.invariants

import data.opus_publica.constitution.invariants.INV_I_02

# Deny if any published article version has zero DOIs.
deny[msg] {
    some v
    v := input.published_versions[_]
    v.state == "published"
    count([d | d := input.dois[_]; d.article_version_id == v.id]) == 0
    msg := sprintf(
        "INV-I-02 FAIL: published article version %s has zero DOIs (expected 1)",
        [v.id]
    )
}

# Deny if any published article version has more than one DOI.
deny[msg] {
    some v
    v := input.published_versions[_]
    v.state == "published"
    count([d | d := input.dois[_]; d.article_version_id == v.id]) > 1
    msg := sprintf(
        "INV-I-02 FAIL: article version %s has %d DOIs (expected 1)",
        [v.id, count([d | d := input.dois[_]; d.article_version_id == v.id])]
    )
}
```

```
}
```

3.2 Data and Input

Every OPA decision takes two inputs: the data document and the input document. The data document is the long-lived, slowly-changing context: the Constitution's provisions, the canonical state-machine definition, the list of accepted licenses, the canonical field schema. In Opus Publica, the data document is the Machine-Readable Constitution (constitution.yaml and its sub-files), loaded into OPA at evaluation time via `--data mrc/`. The data document is what a policy refers to when it writes `data.opus_publica.constitution.invariants.INV_I_02`. The input document is the per-evaluation runtime artifact: the YAML or JSON that represents the state being evaluated. In CI, the input is the evidence file produced by the criterion's verification method. In production, the input is the request being authorized or the configuration being validated.

The discipline of separating data from input is fundamental to writing reusable Rego. A policy that hard-codes a threshold (e.g., 0.95) is brittle; a policy that reads the threshold from data (`data.opus_publica.constitution.thresholds.doi_resolution_min`) is robust to amendment. A policy that hard-codes a list of accepted licenses is wrong the moment the Constitution adds a license; a policy that reads the list from data is correct the moment the MRC is updated. The engineer's default SHALL be: if a value comes from the Constitution, read it from data; if a value comes from the runtime artifact, read it from input.

3.3 The Deny Pattern

Opus Publica adopts the deny pattern as the single convention for expressing policy decisions. A deny rule has the form `deny[msg] { body }` where `body` is a conjunction of expressions that, when all true, causes the rule to fire and emit the message `msg`. The `deny[msg]` form is a partial set rule: `deny` is the name of a set, and each firing of the rule adds `msg` to that set. Evaluating `data.opus_publica.invariants.deny` returns the set of all messages emitted by all fired deny rules. An empty set is PASS; a non-empty set is FAIL. This convention is uniform across all Opus Publica policies, which means a single CI step (`opa eval 'data.opus_publica..deny'` or `conftest test`) can evaluate any policy and interpret the result.

Why deny, not allow?

The deny pattern is preferred because deny rules are easy to compose: two deny rules in the same package both fire independently, and the union of their messages is the policy's decision. The allow pattern is harder to compose: two allow rules express two paths to permission, and the policy's decision is allow if either fires. Deny is also closed-world: anything not denied is permitted, which matches the Constitution's structure (provisions enumerate what MUST NOT happen, not what MAY).

3.4 Built-in Functions

Rego provides a rich library of built-in functions. The Opus Publica policy author uses the following built-ins frequently; the full catalog is at openpolicyagent.org/docs/latest/policy-reference. Each built-in is a function evaluated by the OPA engine; the policy author does not implement them.

Built-in	Purpose	Example
<code>count(x)</code>	Number of elements in array, object, or set.	<code>count(input.dois) == 1</code>
<code>concat(sep, xs)</code>	Join an array of strings with a separator.	<code>concat(",", [a, b, c])</code>
<code>sprintf(fmt, args)</code>	Format a string (printf-style).	<code>sprintf("%s has %d DOIs", [v.id, n])</code>
<code>regex.match(re, s)</code>	Test whether a string matches a regular expression.	<code>regex.match("^10\\.\\.\\d+/.*\$", doi)</code>
<code>time.now_ns()</code>	Current wall-clock time in nanoseconds since epoch.	<code>now := time.now_ns()</code>
<code>object.get(o, k, default)</code>	Look up key <code>k</code> in object <code>o</code> , returning default if <code>k</code> is absent.	<code>object.get(v, "state", "draft")</code>
<code>lower(s) / upper(s)</code>	Case conversion for strings.	<code>lower(input.license) == "cc-by"</code>
<code>contains(s, sub)</code>	Test whether string <code>s</code> contains substring <code>sub</code> .	<code>contains(msg, "FAIL")</code>
<code>to_number(s)</code>	Convert a string to a number.	<code>to_number(input.version)</code>
<code>walk(x)</code>	Yield (path, value) pairs for every node in a tree; useful for deep searches.	<code>some p, v; walk(input)[[p, v]]</code>

3.5 Aggregation and Comprehension

The array comprehension `[x | y]` is the most-used aggregation construct in Opus Publica Rego. The comprehension reads: “produce the array of all `x` such that `y` holds.” The body `y` is a conjunction of expressions, exactly as in a rule body; `x` is the expression that produces each element. Comprehensions are how a policy filters, projects, and aggregates input data. The example in §3.1 uses an array comprehension to count the DOIs for a given article version: `count([d | d := input.dois[_]; d.article_version_id == v.id])`. The comprehension iterates over `input.dois`, binds `d` to each element, filters to those whose `article_version_id` matches `v.id`, and produces the array of matching `d` values; `count` then takes the length of that array.

Set comprehensions `{x | y}` and object comprehensions `{k: v | y}` are analogous, producing sets and objects respectively. The engineer SHALL prefer comprehensions over imperative loops (which Rego does

not have) and over hand-rolled recursion (which Rego supports but which is harder to read). The pattern “filter with a comprehension, then aggregate with count or sum” covers the large majority of Opus Publica's aggregation needs; the engineer who internalizes this pattern can express most provisions concisely.

3.6 Negation and the not Keyword

Rego's not keyword implements negation-as-failure under the closed-world assumption. The expression not p means “p is not derivable from the policy and the data,” which, in the closed world, is equivalent to “p is false.” The engineer new to Rego SHALL internalize this: not does not mean “logically false”; it means “not provable.” The distinction matters when p is itself a negation: not not p is not the same as p in classical logic, but in Rego's closed-world semantics it usually is. Opus Publica policy authors use not primarily to assert the absence of a fact: “there is no DOI for this article version” is written some d; d := input.dois[_]; d.article_version_id == v.id; that is the positive form, and the negative form (used in a deny rule) is not some d; d := input.dois[_]; d.article_version_id == v.id.

```
# Deny if a published article version has NO DOI.
deny[msg]
{
    some v
    v
    v.state := input.published_versions[_]
    v.state == "published"
    not has_doi(v.id)
    msg := sprintf(
        "INV-I-02 FAIL: published article version %s has zero DOIs (expected 1)",
        [v.id]
    )
}

# Helper rule: true iff at least one DOI references the given article version.
has_doi(version_id)
{
    some d
    d
    d := input.dois[_]
    d.article_version_id == version_id
}
```

3.7 Modules and Imports

A Rego file is a module. Modules are combined into a single policy by loading them all into OPA (opa eval --data policies/) or Conftest (--policy policies/). Modules do not need to declare their dependencies explicitly; if module A defines package opus_publica.util and module B imports data.opus_publica.util.foo, the import succeeds as long as both modules are loaded. Opus Publica uses a small set of utility modules to share logic across policies: util.rego (string helpers, DOI validation), time.rego (time arithmetic, evidence-age computation), and schema.rego (canonical field presence

checks). Policy authors SHALL place shared logic in a util module rather than copy-pasting it across policies; the reviewer SHALL reject pull requests that duplicate logic already present in a util module.

Imports in Rego are of two kinds: built-in imports (e.g., `future.keywords`) and data imports (e.g., `data.opus_publica.constitution.invariants.INV_I_02`). The `future.keywords` import is REQUIRED in current OPA versions to enable the `in`, `every`, and `contains` keywords; every Opus Publica policy file SHALL begin with `import future.keywords` if it uses any of these. Data imports are optional: a policy can always reference data by its full path. The data import is a convenience that makes the policy more readable; it does not affect evaluation.

Chapter 4 — Authoring a Policy from a Constitutional Provision

4.1 The Authoring Workflow

Authoring a Rego policy from a constitutional provision is a seven-step procedure. The procedure is binding: a policy that skips a step is a policy whose author cannot demonstrate that the policy encodes the provision. The seven steps are:

5. **Step 1** — read the provision in the Machine-Readable Constitution and extract its fields (ID, statement, predicate, evidence schema);
6. **Step 2** — translate the predicate to one or more Rego rules, using the patterns in Chapter 5 as templates;
7. **Step 3** — handle the input data: write the policy against the evidence schema the CI job will provide;
8. **Step 4** — write descriptive deny messages that include the provision ID and a human-readable explanation;
9. **Step 5** — write the test file with at least one PASS case and at least one FAIL case;
10. **Step 6** — run the test locally with opa test and conftest test until both pass;
11. **Step 7** — submit the pull request; CI runs the policy in the appropriate mode (report-only in Phase 1, blocking in Phase 3) and the reviewer reviews the policy against the provision.

The rest of this chapter walks each step in detail, then presents a worked example: the encoding of provision INV-I-02 (exactly one DOI per published article version) from start to finish.

4.2 Step 1: Read the Provision

Every constitutional provision is recorded in the Machine-Readable Constitution as a YAML object. The policy author begins by reading that object. The object's fields are: id (the stable provision identifier), statement (the human-readable statement of the provision), predicate (the formal predicate, in a pseudo-Datalog syntax that translates directly to Rego), evidence_schema (the JSON Schema that the CI job's input MUST satisfy), and source (the Constitution chapter and section). The policy author SHALL NOT begin writing Rego until they have read all five fields. The predicate is the contract between the provision and the policy: if the policy does not express the predicate, the policy is wrong.

4.3 Step 2: Translate the Predicate to Rego

The formal predicate in the MRC is a pseudo-Datalog expression that translates almost line-for-line into Rego. The translation is mechanical: COUNT becomes count, FOR ALL becomes the some binding plus the implicit universal of the rule body, EXISTS becomes a non-empty comprehension, = stays =, != stays !=, AND becomes the conjunction (newline or semicolon), OR becomes a separate rule. The common patterns are catalogued in Chapter 5; the engineer who has internalized the patterns can translate most

predicates in minutes. The translation SHALL preserve the predicate's structure: a conjunctive predicate becomes a single deny rule with multiple body expressions; a disjunctive predicate becomes multiple deny rules (one per disjunct). The engineer SHALL NOT collapse a disjunctive predicate into a single rule with complex boolean logic; the multiple-rule form is more readable and produces more specific deny messages.

4.4 Step 4: Handle the Input Data

The policy's input is the evidence file the CI job produces. The evidence file's schema is declared in the MRC's `evidence_schema` field. The policy author writes the policy against that schema: `input.published_versions`, `input.dois`, `input.crossref_reconciliation`, and so on. The policy author SHALL NOT assume fields that the schema does not guarantee; if a field is optional in the schema, the policy SHALL use `object.get` to access it with a safe default. The CI job MUST validate the evidence file against the schema before invoking the policy; this is a separation of concerns: schema validation is JSON Schema's job, semantic evaluation is Rego's job. A policy that re-implements schema validation is a policy that duplicates the schema and will drift from it.

4.5 Step 4 (continued): Write Descriptive Deny Messages

Every deny message emitted by an Opus Publica policy SHALL conform to the format: `<PROVISION-ID> FAIL: <human-readable explanation>`. The provision ID is the stable identifier the Constitution assigns; the human-readable explanation is a single sentence that states what is wrong, in terms a reviewer who has not read the policy can understand. The message SHALL include the identifiers of the runtime objects involved (the article version ID, the DOI value, the contract ID) so that the reviewer can locate the offending object without re-running the policy. The message SHALL NOT include remediation instructions; remediation is the responsibility of the implementer, not the policy. The message format is enforced by a meta-policy (`policies/_meta/message_format.rego`) that runs in CI and fails any pull request whose deny messages do not match the format.

#	GOOD	deny	message
msg	:=	sprintf(	
	"INV-I-02	FAIL: article version %s has %d DOIs (expected 1)",	n]
)		[v.id,	
#	BAD	deny	message (no provision ID)
msg	:=	sprintf("too many DOIs on %s",	[v.id])
#	BAD	deny	message (no identifier of the offending object)
msg	:=	"INV-I-02	FAIL: an article version has too many DOIs"
#	BAD	deny	message (remediation instruction)
msg	:=	sprintf(	
	"INV-I-02	FAIL: article version %s has %d DOIs; remove all but one",	

```
) [v.id, n]
```

4.6 Step 5: Write the Test

The test file lives at `tests/<PROVISION-ID>_test.rego`. The `_test` suffix is mandatory: the OPA test runner discovers test files by suffix. The test file declares the same package as the policy it tests. Each test is a rule whose name begins with `test_`; the rule body uses the `assert` pattern (`assert <boolean>`) to express the test's expectation. A test passes if its body's assertion holds; it fails if the assertion does not hold or if any expression in the body raises an error. Every policy SHALL have at least one PASS test (the policy does not fire on valid input) and at least one FAIL test (the policy does fire on invalid input). The PASS test is the proof that the policy is not over-broad; the FAIL test is the proof that the policy is not vacuous. Chapter 6 specifies the test framework in detail.

4.7 Step 6: Run Locally

The policy author runs two commands in their local clone, repeatedly, until both pass. The first is `opa test`, which runs the Rego test framework: `opa test policies/INV-I-02.rego tests/INV-I-02_test.rego`. The second is `conftest test`, which runs the policy against fixture YAML/JSON files: `conftest test --policy policies/ evidence/INV-I-02/sample-pass.yaml evidence/INV-I-02/sample-fail.yaml`. The first command verifies the policy's logic against the test cases; the second verifies the policy's integration with Conftest against realistic evidence. Both SHALL pass before the author opens a pull request. The CI workflow runs the same two commands; a local pass is the strongest predictor of a CI pass.

4.8 Step 7: Submit the Pull Request

The pull request includes four files: the policy (`policies/INV-I-02.rego`), the test (`tests/INV-I-02_test.rego`), the evidence fixtures (`evidence/INV-I-02/sample-pass.yaml` and `evidence/INV-I-02/sample-fail.yaml`), and (if the policy is new) the criterion row in the MRC's `criteria.yaml`. The pull request description SHALL link to the provision in the Constitution and SHALL state, in one sentence, the predicate the policy encodes. The CI workflow `.github/workflows/policy-test.yml` runs on the PR; it executes `opa test` and `conftest test` and reports the result as a status check. During Phase 1 of the RC2 migration, the status check is informational (a FAIL does not block merge); during Phase 3, the status check is required (a FAIL blocks merge). The reviewer SHALL verify that the policy's deny rules correspond exactly to the provision's predicate, that the test suite covers both PASS and FAIL cases, and that the deny messages conform to the format. The reviewer SHALL NOT approve the PR until these verifications pass.

4.9 Worked Example: INV-I-02

This section demonstrates the seven-step procedure on the constitutional provision INV-I-02 (exactly one DOI per published article version). The provision is from Volume II, Chapter 26, of the Engineering Constitution, and is one of the 38 engineering invariants. The MRC entry for the provision is:

```
# mrc/invariants/INV-I-02.yaml
id: INV-I-02
statement: |
    Exactly one DOI is registered for every published article version.
predicate: |
    FOR ALL v IN published_versions WHERE v.state = "published":
        COUNT(d IN dois WHERE d.article_version_id = v.id) = 1
        AND
        FOR ALL d IN dois WHERE d.article_version_id = v.id:
            crossref_reconciliation[d.value].resolved = true
evidence_schema: mrc/schemas/INV-I-02-evidence.json
source: "Vol II Ch 26 §26.2"
severity_on_fail: SEV-1
gating: GATE
```

The predicate has two conjuncts: a count constraint (exactly one DOI per published version) and a resolvability constraint (every DOI resolves via Crossref). The policy therefore has two deny rules: one that fires when the count is not 1, and one that fires when a DOI does not resolve. The Rego policy is:

```
# policies/INV-I-02.rego
# Encoding of constitutional provision INV-I-02.
# Source: Vol II Ch 26 §26.2 (Engineering Invariants).
# PASS: every published article version has exactly one DOI, and that DOI
# resolves via Crossref.

package opus_publica.invariants

import future.keywords.in
import data.opus_publica.constitution.invariants.INV_I_02

# Conjunct 1: every published article version has exactly one DOI.
deny[msg] {
    some v in input.published_versions
        v.state == "published"
    n := count([d | d := input.dois[_]; d.article_version_id == v.id])
    n != 1
    msg := sprintf(
        "INV-I-02 FAIL: article version %s has %d DOIs (expected 1)",
        [v.id, n]
    )
}
```

```

}

# Conjunct 2: every DOI registered for a published article version resolves
# via Crossref.
deny[msg] {
    some v in input.published_versions
        v.state == "published"
        some d in input.dois
            d.article_version_id == v.id
        not input.crossref_reconciliation[d.value].resolved
        msg := sprintf(
            "INV-I-02 FAIL: DOI %s (article version %s) does not resolve via Crossref",
            [d.value, v.id]
        )
}

```

The test file covers both conjuncts with a PASS case and two FAIL cases (one per conjunct):

```

# tests/INV-I-02_test.rego
package opus_publica.invariants

import future.keywords.in

# PASS: one published version with one resolving DOI.
test_pass_single_doi_resolves {
    count(deny) == 0 with input as {
        "published_versions": [{"id": "v1", "state": "published"}],
        "dois": [{"article_version_id": "v1", "value": "10.57939/ABC-001"}],
        "crossref_reconciliation": {"10.57939/ABC-001": {"resolved": true}}
    }
}

# FAIL: published version with two DOIs.
test_fail_two_dois {
    some msg in deny with input as {
        "published_versions": [{"id": "v1", "state": "published"}],
        "dois": [
            {"article_version_id": "v1", "value": "10.57939/ABC-001"},
            {"article_version_id": "v1", "value": "10.57939/ABC-002"}
        ],
        "crossref_reconciliation": {
            "10.57939/ABC-001": {"resolved": true},
            "10.57939/ABC-002": {"resolved": true}
        }
    }
    contains(msg, "has 2 DOIs")
}

```

```

#      FAIL:      published      version      with      a      non-resolving      DOI.
test_fail_unresolvable_doi      {
    some      msg      in      deny      with      input      as      {
        "published_versions":      [{"id":      "v1",      "state":      "published"}],
        "dois":      [{"article_version_id":      "v1",      "value":      "10.57939/ABC-DEAD"}],
        "crossref_reconciliation":      {"10.57939/ABC-DEAD":      {"resolved":      false}}
    }
    contains(msg,      "does      not      resolve      via      Crossref")
}

#      PASS:      draft      (unpublished)      version      with      zero      DOIs      is      not      a      violation.
test_pass_draft_version_no_doi      {
    count(deny)      ==      0      with      input      as      {
        "published_versions":      [{"id":      "v1",      "state":      "draft"}],
        "dois":      [],
        "crossref_reconciliation":      {}
    }
}

```

The CI integration is in `.github/workflows/policy-test.yml`; the relevant job, shown in Chapter 7, runs opa test on the policy and conftest test on the evidence fixtures. The pull request that introduces this policy will be reviewed against the predicate in the MRC, the deny-message format, and the test coverage. Once merged, the policy is distributed via the OPA bundle to every CI runner and every production sidecar that evaluates INV-I-02. The author's work is done; the policy's work has begun.

Chapter 5 — Common Policy Patterns

The 600+ constitutional provisions of Opus Publica reduce, on close inspection, to a small set of structural patterns: uniqueness, immutability, resolvability, completeness, legality, auditability, idempotency, and threshold enforcement. This chapter catalogues each pattern with a description, a Rego template, a statement of when to use it, and a worked example. The engineer who has internalized these eight patterns can encode the large majority of provisions without designing a new structure for each one.

5.1 Uniqueness

The uniqueness pattern asserts that exactly one canonical record exists for a given key. It is the pattern of INV-I-01 (exactly one canonical article per work), INV-I-02 (exactly one DOI per published version), and ARCH-003 (exactly one canonical record per published artifact). The Rego template is a deny rule that counts the records matching the key and fires when the count is not 1. The pattern's complexity comes from the count, which uses an array comprehension, and from the key extraction, which depends on the input's shape.

```
#                                Template:                                uniqueness
deny[msg]                                                                {
    some                                                                    k
    records := [r | r := input.records[_]; r.key == k]                    count(records) != 1
    msg := sprintf(                                                         (
        "<PROV-ID> FAIL: key %s has %d records (expected 1)",             [k, count(records)]
    )
}
```

When to use: any provision that asserts a one-to-one or one-to-many relationship where the “one” side is a single canonical record. Worked example: INV-I-01, exactly one canonical article per work, where the key is `work_id` and the records are articles with `is_canonical = true`. The policy fires when `count([a | a := input.articles[_]; a.work_id == w; a.is_canonical])` is not 1 for any work `w`.

5.2 Immutability

The immutability pattern asserts that the hash of a published artifact does not change between two observations. It is the pattern of INV-A-01 (the content hash of a published artifact is immutable), of INV-P-03 (the deposit hash of a CLOCKSS deposit is immutable), and of the audit-record integrity rule. The Rego template compares the hash in the before-state to the hash in the after-state and fires when they differ. The pattern's complexity comes from the join: the policy must match the same artifact in two observations, which requires a shared identifier.

```

#                                Template:                                immutability
deny[msg]                        {
                                some      a      in      input.before
                                some      b      in      input.after
                                a.id      ==      b.id
                                a.hash    !=      b.hash
                                msg       :=      sprintf(
"<PROV-ID>  FAIL:  artifact  %s  hash  changed  from  %s  to  %s",
                                [a.id,      a.hash,      b.hash]
                                )
}

```

When to use: any provision that asserts the immutability of a published, deposited, or audited artifact. Worked example: INV-A-01, where before is the snapshot at publication time and after is the snapshot at the current verification; the policy fires when any artifact's content_hash differs between the two snapshots.

5.3 Resolvability

The resolvability pattern asserts that an identifier resolves via an external service. It is the pattern of INV-I-02's second conjunct (every DOI resolves via Crossref), of INV-A-04 (every ORCID iD resolves via the ORCID Public API), and of the contract endpoint resolvability rule. The Rego template fires when the external service's response does not indicate resolution. The pattern's complexity comes from the integration: the policy does not perform the HTTP probe itself; it evaluates the probe's result, which is in the input as a pre-computed reconciliation object. The probe is performed by the CI job before the policy runs.

```

#                                Template:                                resolvability
deny[msg]                        {
                                some      id      in      input.identifiers
                                not      input.reconciliation[id.value].resolved
                                msg       :=      sprintf(
"<PROV-ID>  FAIL:  identifier  %s  does  not  resolve  via  %s",
                                [id.value,      id.authority]
                                )
}

```

When to use: any provision that asserts an identifier's external resolvability. Worked example: INV-I-02 conjunct 2, where identifiers is the list of DOIs and reconciliation is the Crossref REST API's response, with resolved = true if the DOI is registered and resolvable.

5.4 Completeness

The completeness pattern asserts that no record is an orphan: every reference in the data has a target. It is the pattern of INV-I-03 (every article references a canonical work), of INV-A-02 (every audit record references an event), and of the canonical-field referential integrity rule. The Rego template fires when a record references a target that is not in the data. The pattern's complexity comes from the membership test, which uses a set comprehension to build the set of valid targets and the `in` keyword to test membership.

```
# Template: completeness (no orphans)
deny[msg] {
    some r in input.records
    valid_targets := {t.id | t := input.targets[_]}
    not r.target_id in valid_targets
    msg := sprintf(
        "<PROV-ID> FAIL: record %s references missing target %s",
        [r.id, r.target_id]
    )
}
```

When to use: any provision that asserts referential integrity. Worked example: INV-I-03, where records is the list of articles and targets is the list of canonical works; the policy fires when any article's `work_id` is not in the set of canonical work IDs.

5.5 Legality

The legality pattern asserts that a state transition is permitted by the state machine. It is the pattern of the publication state machine (13 states, 17 transitions), of the audit-record state machine, and of the certification state machine. The Rego template looks up the transition in the data document (the state-machine definition) and fires when the transition is not in the permitted set. The pattern's complexity comes from the data lookup, which requires the policy to navigate the state-machine definition in the MRC.

```
# Template: legality (state-machine transition)
deny[msg] {
    some t in input.transitions
    permitted := data.opus_publica.constitution.state_machine[
        t.from_state
    ].permitted_transitions
    not t.to_state in permitted
    msg := sprintf(
        "<PROV-ID> FAIL: illegal transition %s -> %s (permitted: %s)",
        [t.from_state, t.to_state, concat(", ", permitted)]
    )
}
```



```
}
```

When to use: any provision that asserts the legality of a state transition. Worked example: PUB-001, where transitions is the list of state changes observed in the audit log and the state machine is defined in mrc/state_machine.yaml. The policy fires when any transition's to_state is not in the permitted set for its from_state.

5.6 Auditability

The auditability pattern asserts that every audit record contains all mandatory fields and that the audit chain is verifiable. It is the pattern of INV-A-02 (every audit record has the seven mandatory fields) and of the audit chain integrity rule (every audit record's prev_hash equals the hash of the previous record). The Rego template fires when a mandatory field is missing or when the chain is broken. The pattern's complexity comes from the field-presence check, which uses object.get, and from the chain verification, which requires the policy to sort the records by sequence number and compare adjacent hashes.

```
#           Template:           auditability           (field           completeness)
deny[msg]                                     {
    some r in input.audit_records
    some field in data.opus_publica.constitution.audit.mandatory_fields
    not object.get(r, field, "") != ""
    msg := sprintf(
        "<PROV-ID> FAIL: audit record %s is missing mandatory field %s",
        [r.id, field]
    )
}

#           Template:           auditability           (chain           integrity)
deny[msg]                                     {
    some i
    i < count(input.audit_records) - 1
    r := input.audit_records[i]
    r_next := input.audit_records[i + 1]
    r_next.prev_hash != r.hash
    msg := sprintf(
        "<PROV-ID> FAIL: audit chain broken between %s and %s",
        [r.id, r_next.id]
    )
}
```

When to use: any provision that asserts audit-record completeness or chain integrity. Worked example: INV-A-02, where audit_records is the list of audit records and mandatory_fields is the seven-field list

from the Constitution; the policy fires when any record is missing a field, and the chain-integrity rule fires when any adjacent pair does not satisfy prev_hash = hash.

5.7 Idempotency

The idempotency pattern asserts that a retry of an operation with the same idempotency key returns the same result as the original. It is the pattern of the submission idempotency rule (every submission retry with the same key returns the same submission ID) and of the deposit idempotency rule. The Rego template groups operations by idempotency key and fires when the results in a group differ. The pattern's complexity comes from the grouping, which uses a set comprehension to collect the distinct keys and an array comprehension to filter the operations in each group.

```
# Template: idempotency
deny[msg]
    {
        some key
        ops := [o | o := input.operations[_]; o.idempotency_key == key]
        count(ops) > 1
        results := {o.result | o := ops[_]}
        count(results) != 1
        msg := sprintf(
            "<PROV-ID> FAIL: idempotency key %s returned %d distinct results",
            [key, count(results)]
        )
    }
```

When to use: any provision that asserts the idempotency of a retryable operation. Worked example: SUB-IDEMP-001, where operations is the list of submission operations observed in the audit log and idempotency_key is the client-supplied key; the policy fires when any key has more than one operation with distinct results.

5.8 Threshold Enforcement

The threshold enforcement pattern asserts that a measured value satisfies a constitutional threshold. It is the pattern of the SLO rules (latency p95 ≤ 300 ms, availability ≥ 99.9%), of the error-budget rules (error budget not exhausted), and of the reconciliation-latency rules (reconciliation completed within 1 hour). The Rego template reads the threshold from the data document, reads the measurement from the input, and fires when the threshold is violated. The pattern's complexity comes from the threshold's direction (some thresholds are upper bounds, some are lower bounds) and from the comparison's semantics (some thresholds are inclusive, some exclusive).

```
# Template: threshold enforcement (upper bound)
deny[msg]
    {
        some m in input.measurements
        threshold := data.opus_publica.constitution.thresholds[m.metric].upper_bound
```

```

                                m.value      >      threshold
                                msg           :=      sprintf(
"<PROV-ID>  FAIL:  metric  %s  value  %v  exceeds  threshold  %v",
                                [m.metric,      m.value,      threshold]
                                )
}

# Template: threshold enforcement (lower bound, e.g. error budget remaining)
deny[msg]
    some      m      in      input.measurements
    threshold := data.opus_publica.constitution.thresholds[m.metric].lower_bound
                                m.value      <      threshold
                                msg           :=      sprintf(
"<PROV-ID>  FAIL:  metric  %s  value  %v  below  threshold  %v",
                                [m.metric,      m.value,      threshold]
                                )
}

```

When to use: any provision that asserts a numerical bound. Worked example: SLO-LAT-001, where `measurements` is the latency probe's result and `threshold` is `data.opus_publica.constitution.thresholds.slo_latency_p95_ms.upper_bound = 300`. The policy fires when any measurement exceeds the threshold; the CI job that produces the measurement runs the policy and emits the OSCAL-AR fragment that becomes part of the release's certification record.

Chapter 6 — Testing Policies

6.1 The Rego Test Framework

Rego ships with a built-in test framework. A test is a rule whose name begins with `test_`; the test passes if the rule body is true (i.e., the rule fires) and fails if the rule body is false or raises an error. The framework is invoked by the `opa test` command, which discovers test files by the `_test.rego` suffix and executes every `test_` rule in them. The framework supports the `with` keyword, which overrides the input or data for the duration of a single expression; this is the mechanism by which a test provides its fixture. The framework is sufficient for Opus Publica's needs; no external test framework is required.

```
# tests/example_test.rego
package opus_publica.invariants

import future.keywords.in

test_pass_case {
    count(deny) == 0 with input as {"x": 1}
}

test_fail_case {
    some msg in deny with input as {"x": 0}
    contains(msg, "FAIL")
}
```

6.2 Test Fixtures

Test fixtures are the input data the test rules provide via the `with` keyword. Fixtures come in two forms: inline (the input is written directly in the test rule, as in the example above) and external (the input is read from a YAML or JSON file in the `evidence/` directory). Inline fixtures are appropriate for small, self-contained tests; external fixtures are appropriate for larger, more realistic inputs and for inputs that are also used by Conftest (which reads files, not inline data). Opus Publica uses both: every policy has at least two inline fixtures (one PASS, one FAIL) for clarity, and at least two external fixtures (one PASS, one FAIL) in `evidence/<PROV-ID>/` for realism and Conftest integration. The external fixtures are version-controlled alongside the policy.

6.3 Test Coverage

Every Opus Publica policy SHALL have a test suite that satisfies the following coverage requirements. The requirements are enforced by a meta-policy (`policies/_meta/test_coverage.rego`) that runs in CI and fails any pull request whose test suite does not satisfy them.

12. **PASS case.** at least one PASS case — the policy does not fire on valid input. The PASS case is the proof that the policy is not over-broad.
13. **FAIL case per rule.** at least one FAIL case per deny rule — the policy fires on the invalid input the rule is designed to catch. The FAIL case is the proof that the rule is not vacuous.
14. **Edge case.** at least one edge case per rule — the policy behaves correctly on boundary inputs (empty arrays, single-element arrays, maximum-size arrays, missing optional fields).
15. **Regression case.** a regression case for every bug previously found in the policy — the test that would have caught the bug. The regression case SHALL be annotated with a comment linking to the bug ticket.

6.4 Conftest Test

The Conftest test is the second leg of the testing regime. Where opa test verifies the policy's logic against inline fixtures, conftest test verifies the policy's integration with the CI evidence format against external YAML/JSON files. The Conftest command is: `conftest test --policy policies/ evidence/<PROV-ID>/*.yaml`. Conftest evaluates every file in the evidence directory against every policy in the policies directory, and reports PASS or FAIL per file. The evidence fixtures are named to indicate their expected result (sample-pass.yaml, sample-fail.yaml, edge-empty.yaml, edge-single.yaml, regression-001.yaml); a CI step asserts that the *-pass*.yaml files PASS and the *-fail*.yaml files FAIL, and fails the build if the actual results do not match the expected.

6.5 Property-Based Testing

Property-based testing is OPTIONAL for provisions whose predicate has a parameter space that admits random generation. The Rego test framework does not natively support property-based testing, but the fuzzer go-quickcheck can be wrapped in a small Go program that generates random inputs, evaluates the policy via the OPA Go library, and asserts the policy's invariants. Property-based testing is RECOMMENDED for policies whose predicate is a mathematical invariant (e.g., the audit chain hash is always the SHA-256 of the previous record) and OPTIONAL for policies whose predicate is a structural constraint (e.g., exactly one DOI per version). The engineering cost of property-based testing is not justified for every policy; it is justified for the small subset whose failure mode is silent corruption.

6.6 CI Integration

The CI workflow that runs the policy tests is `.github/workflows/policy-test.yml`. The workflow triggers on every pull request that touches the policies/ or tests/ directories. The workflow has three jobs: opa-test (runs opa test on the entire policies/ + tests/ tree), conftest-test (runs conftest test on every evidence/ fixture), and meta-check (runs the meta-policies that enforce test coverage and deny-message format). Each job is a required status check; a failure in any job blocks the pull request from merging (Phase 3) or surfaces a warning (Phase 1). The full workflow YAML is in Chapter 7.

6.7 Worked Example: The INV-I-02 Test File

The test file for the INV-I-02 policy (Chapter 4) is the canonical example of the testing regime. The file has four tests: a PASS case for a single resolving DOI, a FAIL case for two DOIs (covering conjunct 1), a FAIL case for a non-resolving DOI (covering conjunct 2), and a PASS case for a draft version with zero DOIs (covering the edge case where the provision does not apply). Each test is self-contained: the input is inline, the assertion is a single expression, and the test name states what it tests. The file is short (under 60 lines) and complete; the engineer who reads it can verify, by inspection, that the policy behaves correctly on each of the four inputs. This is the standard the reviewer SHALL apply to every test file: short, complete, and verifiable by inspection.

Chapter 7 — CI/CD Integration

7.1 The policy-test Workflow

The policy-test workflow runs on every pull request that touches the policies/ or tests/ directories. The workflow has three jobs (opa-test, conftest-test, meta-check), each of which runs on a separate GitHub Actions runner for parallelism. The workflow installs OPA and Conftest via their official GitHub releases, checks out the repository, and runs the relevant commands. The workflow's status check is a required check (Phase 3) on the main branch's branch protection rule.

```
# .github/workflows/policy-test.yml
name: policy-test

on:
  pull_request:
    paths:
      - 'policies/**'
      - 'tests/**'
      - 'evidence/**'
  workflow_dispatch:
    - '.github/workflows/policy-test.yml'

permissions:
  contents: read
  checks: write

jobs:
  opa-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install OPA
        run: |
          curl -L -o opa https://openpolicyagent.org/downloads/latest/opa_linux_amd64_static
          chmod +x opa && sudo mv opa /usr/local/bin/
      - name: Run opa test
        run: opa test --v0-compatible policies/ tests/

  conftest-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Conftest
        run: |
          curl -L -o conftest
          https://github.com/open-policy-agent/conftest/releases/latest/download/
          conftest_0.45.0_Linux_x86_64.tar.gz
```

```

tar xzf conftest && sudo mv conftest /usr/local/bin/
- name: Run conftest test on evidence fixtures
  run: |
    conftest test --policy policies/ evidence/**/*.yaml \
      --reporter json > conftest-results.json
python3 scripts/assert-conftest-expectations.py conftest-results.json

  meta-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install OPA
        run: |
          curl -L -o opa https://openpolicyagent.org/downloads/latest/opa_linux_amd64_static
          chmod +x opa && sudo mv opa /usr/local/bin/
- name: Run meta-policies (coverage, message format)
  run: opa eval --data policies/ --data tests/ \
    --format pretty 'data.opus_publica._meta.deny'

```

7.2 The conftest-verify Workflow

The conftest-verify workflow runs on every pull request that touches the `contracts/` directory (where the system contract YAMLs live). The workflow evaluates every changed contract against the relevant Rego policies and reports violations. Where policy-test verifies the policies themselves, conftest-verify verifies the contracts the policies govern. The workflow uses git diff to identify the changed files and passes only those files to Conftest, keeping the job fast on large pull requests.

```

# .github/workflows/conftest-verify.yml
name: conftest-verify

on:
  pull_request:
    paths:
      - 'contracts/**'
      - 'policies/**'
  - '.github/workflows/conftest-verify.yml'

permissions:
  contents: read
  pull-requests: write

jobs:
  verify-contracts:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

```



```

with:
  fetch-depth: 0
- name: Install Conftest
  run: |
    curl -L -o conftest.tgz
    https://github.com/open-policy-agent/conftest/releases/latest/download/
    conftest_0.45.0_Linux_x86_64.tar.gz
    tar xzf conftest.tgz && sudo mv conftest /usr/local/bin/
- name: Identify changed contracts
  id: changes
  run: |
    FILES=$(git diff --name-only origin/${{ github.base_ref }} HEAD \
      -- 'contracts/*.yaml' | tr '\n' ' ')
    echo "files=$FILES" >> $GITHUB_OUTPUT
- name: Run Conftest on changed contracts
  if: steps.changes.outputs.files != ''
  run: |
    conftest test --policy policies/ ${{ steps.changes.outputs.files }} \
      --reporter json > conftest-results.json || true
    python3 scripts/post-conftest-comment.py conftest-results.json \
      ${{ github.event.pull_request.number }}

```

7.3 The nightly-criterion Workflow

The nightly-criterion workflow runs every criterion's policy on a scheduled cron. Where policy-test runs policies on pull-request evidence, nightly-criterion runs policies on the live state of the platform: it pulls fresh evidence from the production API, writes it to evidence/nightly/, and evaluates every policy against the fresh evidence. The workflow's purpose is to catch violations that were introduced not by a code change (which policy-test would catch) but by a data change (which only nightly verification catches). The workflow opens a ticket on every FAIL and posts a summary to the governance dashboard.

```

# .github/workflows/nightly-criterion.yml
name: nightly-criterion

on:
  schedule:
    - cron: '0 2 * * *' # 02:00 UTC daily
    workflow_dispatch:

permissions:
  contents: read
  issues: write

jobs:
  collect-evidence:

```

```

                                runs-on:          ubuntu-latest
                                outputs:
evidence_path:  ${ steps.collect.outputs.evidence_path  }
                                steps:
-               uses:          actions/checkout@v4
  name: Collect fresh evidence from production API
                                env:
    OPUS_API_TOKEN:  ${ secrets.OPUS_API_TOKEN  }
                                run:          |
    python3 scripts/collect-evidence.py --output evidence/nightly/
    echo "evidence_path=evidence/nightly" >> $GITHUB_OUTPUT
                                uses:          actions/upload-artifact@v4
                                with:
      name:          nightly-evidence
      path:          evidence/nightly/

                                evaluate:
                                needs:          collect-evidence
                                runs-on:          ubuntu-latest
                                steps:
-               uses:          actions/checkout@v4
-               uses:          actions/download-artifact@v4
                                with:
      name:          nightly-evidence
      path:          evidence/nightly/
-   name: Install OPA + Conftest
                                run:          |
    curl -L -o opa https://openpolicyagent.org/downloads/latest/opa_linux_amd64_static
    chmod +x opa && sudo mv opa /usr/local/bin/
                                curl -L -o conftest.tgz
https://github.com/open-policy-agent/conftest/releases/latest/download/
conftest_0.45.0_Linux_x86_64.tar.gz
    tar xzf conftest.tgz && sudo mv conftest /usr/local/bin/
-   name: Evaluate every criterion
                                run:          |
    python3 scripts/run-all-criteria.py \
      --policies policies/ \
      --evidence evidence/nightly/ \
      --output oscal-ar/nightly-$(date +%Y-%m-%d).json
-   name: Open tickets on FAIL
                                run:          |
    python3 scripts/open-fail-tickets.py \
      --oscal-ar oscal-ar/nightly-*.json
-   uses:          actions/upload-artifact@v4
                                with:
      name:          nightly-oscal-ar
      path:          oscal-ar/

```

7.4 The release-certification Workflow

The release-certification workflow runs on every merge to the release branch. It is the workflow that produces the release's certification record: it runs every GATE criterion's policy, aggregates the results into an OSCAL Assessment Results document, and uploads the document to the certification repository. The workflow is the operational form of the Production Acceptance Gate (RC1 Certification Checklist Chapter 14): in RC2, the Gate is a GitHub Environment with Required Checks, and this workflow is the source of those checks. A release cannot proceed until every GATE criterion's policy PASSES.

```
# .github/workflows/release-certification.yml
name: release-certification

on:
  push:
    branches: [release/*]

permissions:
  contents: read
  id-token: write
  pull-requests: write

jobs:
  gate-criteria:
    runs-on: ubuntu-latest
    environment:
      name: release-certification
    url: https://github.com/${{ github.repository }}/actions/runs/${{ github.run_id }}
    steps:
      - uses: actions/checkout@v4
      - name: Install OPA
        run: |
          curl -L -o opa https://openpolicyagent.org/downloads/latest/opa_linux_amd64_static
          chmod +x opa && sudo mv opa /usr/local/bin/
      - name: Evaluate all GATE criteria
        run: |
          python3 scripts/run-gate-criteria.py \
            --policies policies/ \
            --criteria mrc/criteria.yaml \
            --gating GATE \
            --output oscal-ar/release-${{ github.sha }}.json
      - name: Verify OSCAL-AR fragment is well-formed
        run: python3 scripts/validate-osc-ar.py \
          oscal-ar/release-${{ github.sha }}.json
      - name: Sign OSCAL-AR with Sigstore (Cosign)
        uses: sigstore/cosign-installer@v3
        - run: |
            cosign sign-blob --yes \
```

```

--output-certificate oscal-ar/release-${{ github.sha }}.pem \
--output-signature  oscal-ar/release-${{ github.sha }}.sig \
                   oscal-ar/release-${{ github.sha }}.json
-   name: Upload to certification repository
      run: |
python3  scripts/upload-certification-artifact.py \
        --repo opus-publica/certification \
        --release ${{ github.ref_name }} \
        --files oscal-ar/release-${{ github.sha }}.*

```

7.5 OSCAL Assessment Results Emission

Every CI job in the CCE emits its result as an OSCAL Assessment Results (OSCAL-AR) JSON fragment. OSCAL is the NIST-led machine-readable format for security control documentation; the Assessment Results layer is the layer that records the result of assessing a control against a system. For Opus Publica, each constitutional criterion is mapped to an OSCAL control, and each CI run that evaluates the criterion's policy emits an OSCAL-AR fragment. The fragment contains: the criterion ID (mapped to an OSCAL control-id), the result (PASS/FAIL/N-A), the observation (the deny messages, if any), the evidence (a reference to the evidence artifact in the certification repository), the timestamp, and the assessor (the CI job's identifier).

```

# oscal-ar/INV-I-02-2026-08-09T02-00-00Z.json
{
  "assessment-results": {
    "uuid": "urn:uuid:...",
    "metadata": {
      "title": "INV-I-02 nightly assessment",
      "last-modified": "2026-08-09T02:00:00Z",
      "version": "2026-08-09T02:00:00Z"
    },
    "results": [{
      "uuid": "urn:uuid:...",
      "description": "Nightly evaluation of criterion CC-INV-I-02-a.",
      "start": "2026-08-09T02:00:00Z",
      "reviewed-controls": {
        "control-selections": [{
          "including-controls": [{
            "control-id": "INV-I-02"
          }]
        }]
      },
      "observations": [{
        "uuid": "urn:uuid:...",
        "title": "Policy evaluation result",
        "methods": ["TEST"],

```

```

        "subjects":      [{
            "uuid":      "urn:uuid:...",
            "type":      "assessment-step",
            "subject-uuid":      "urn:uuid:..."
        }],
        "collect-method":      "AUTOMATED",
        "props":      [
            {"name":      "result",      "value":      "PASS"},
            {"name":      "deny-messages",      "value":      "[]"},
            {"name":      "evidence-uri",
"value":      "s3://opus-cert/2026-08-09/INV-I-02-evidence.json"}
        ]
    }
}
}

```

7.6 Evidence Upload

Every CI job that evaluates a policy uploads its evidence to the certification repository. The certification repository is a versioned, append-only store of evidence artifacts: every artifact has a stable URI, an upload timestamp, and a hash. The upload is performed by a small Python script (`scripts/upload-certification-artifact.py`) that authenticates to the certification repository's API, uploads the artifact, and returns the artifact's URI. The URI is recorded in the OSCOL-AR fragment's `evidence-uri` property, which is the link between the policy's evaluation and the evidence that supports it. The certification repository is the source of truth for evidence: a PASS that lacks an evidence URI is not a PASS, and a FAIL with an evidence URI is a FAIL that can be independently verified by any auditor with access to the repository.

Chapter 8 — Deploy-Time Enforcement (Gatekeeper)

8.1 OPA Gatekeeper Architecture

OPA Gatekeeper is a Kubernetes admission controller that enforces Rego policies at deploy time. The architecture has three components: the Gatekeeper admission webhook (which intercepts every create/update/delete operation against the Kubernetes API server), the Constraint Template (a CustomResourceDefinition that embeds a Rego policy), and the Constraint (a CustomResource that instantiates a Constraint Template with specific parameters). When the API server receives a request, the webhook invokes Gatekeeper; Gatekeeper evaluates every Constraint that matches the request's resource type; Gatekeeper returns admit or deny based on the union of the Constraint evaluations. The architecture separates the policy (in the Constraint Template) from the parameters (in the Constraint), which allows the same policy to be applied with different parameters in different namespaces or clusters.

8.2 Mapping Rego Policies to Gatekeeper Constraints

An Opus Publica policy authored as a deny rule in policies/ can be enforced by Gatekeeper at deploy time, but the mapping is not automatic. The deny rule's body must be wrapped in a Constraint Template, with two modifications. First, the input document is replaced by the admission review object, which Gatekeeper passes as `input.admission.review`. The policy's references to `input` must be rewritten to navigate the admission review (`input.admission.review.object` for the object being admitted). Second, the deny rule is renamed `violation`, which is Gatekeeper's convention for the rule whose messages Gatekeeper reports as admission violations. The original policy in `policies/` remains the source of truth; the Constraint Template is generated from it by a script (`scripts/generate-gatekeeper-templates.py`) that performs the two modifications.

```
# templates/k8srequiredslo-template.yaml
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8srequiredslo
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredSLO
      validation:
        openAPIV3Schema:
          type: object
          properties:
            allowedNamespaces:
              type: array
```

```

items:      {type:      string}
targets:
- target:    admission.k8s.gatekeeper.sh
              rego:      |
              package     k8srequiredslo

              violation[{"msg":      msg}]      {
input.review.kind.kind      ==      "Deployment"
not      has_slo_annotation(input.review.object)
              msg      :=      sprintf(
"SLO-DEPLOY-001 FAIL: Deployment %s/%s has no slo.opuspublica.com/id
annotation",
[input.review.object.metadata.namespace, input.review.object.metadata.name]
              )
              }

              has_slo_annotation(obj)      {
obj.metadata.annotations["slo.opuspublica.com/id"]
              }

```

Once the Constraint Template is applied to the cluster, a Constraint instantiates it with parameters (e.g., the namespaces to which it applies):

```

# constraints/k8srequiredslo-constraint.yaml
apiVersion:      constraints.gatekeeper.sh/v1beta1
kind:      K8sRequiredSLO
metadata:
name:      require-slo-on-all-deployments
spec:
enforcementAction:      deny
match:
kinds:
- apiGroups:      ["apps"]
kinds:      ["Deployment"]
parameters:
allowedNamespaces:
- production
- staging

```

8.3 The Enforcement Modes

Gatekeeper supports three enforcement modes, selected by the enforcementAction field on the Constraint. The modes are:

Mode	Behaviour	When to Use
------	-----------	-------------

deny	The admission request is rejected; the deployment does not occur.	Production enforcement, after the policy has been dryrun-clean for two weeks.
dryrun	The admission request is admitted; the violation is recorded in Gatekeeper's audit log and on the constraint's status.	Initial rollout, parity measurement, and the Phase 1 to Phase 3 cutover.
warn	The admission request is admitted; the API server returns a warning to the caller, which the deployment pipeline logs.	Soft enforcement during a deprecation window; e.g., a policy that becomes deny at the next release.

Opus Publica's convention is that every new Gatekeeper-enforced policy starts in dryrun for two weeks. The two-week window is the parity measurement: the team verifies that the policy fires on real traffic exactly as the test suite predicts. If the dryrun log shows false positives, the policy is revised; if it shows false negatives, the policy is strengthened. After two clean weeks, the policy is switched to deny. The warn mode is used only for deprecation: a policy that becomes deny at the next release is warn in the release before, to give implementers a window to remediate.

8.4 Violation Messages and Audit

Gatekeeper's violation messages are the deny messages of the underlying Rego policy, reformatted as Gatekeeper's violation[{"msg": msg}] structure. The messages appear in the kubectl output when a deployment is denied, in the deployment pipeline's logs, and in Gatekeeper's audit log. The audit log is Gatekeeper's record of every violation observed, whether the constraint was deny or dryrun; the audit log is the source of truth for the parity dashboard. Opus Publica's Gatekeeper audit log is exported to the central log store and queried by the parity dashboard, which tracks, per policy, the count of violations observed in dryrun mode over time. A policy whose dryrun violations are non-zero and unchanging is a policy whose implementers have not yet remediated; a policy whose dryrun violations are zero for two weeks is a policy ready to switch to deny.

8.5 Worked Example: No Deployment Without SLO Defined

The policy SLO-DEPLOY-001 asserts that no Deployment may be admitted to the production or staging namespace without an slo.opuspublica.com/id annotation that references an SLO defined in the MRC. The policy is enforced by Gatekeeper at admission time. The Constraint Template (above) defines the violation rule; the Constraint instantiates it with enforcementAction: deny and parameters that restrict it to production and staging. The policy's test suite (tests/SLO-DEPLOY-001_test.rego) covers the PASS case (a Deployment with a valid SLO annotation), the FAIL case (a Deployment with no annotation), and the edge case (a Deployment in a namespace not in allowedNamespaces, which the policy does not evaluate). The policy's two-week dryrun window produces zero violations, demonstrating that every

production and staging Deployment already has an SLO annotation; the policy is switched to deny and remains in force. A future Deployment that omits the annotation is denied at admission, with a violation message that names the Deployment and the missing annotation; the deployment pipeline fails fast, and the implementer adds the annotation before retrying.

Chapter 9 — Policy Debugging and Operation

9.1 The OPA Decision Log

Every decision an OPA instance makes is recorded in the OPA decision log. The decision log is a structured record of every evaluation: the timestamp, the policy package evaluated, the input document, the decision (the set of deny messages, or the allow boolean), and the decision ID. In Opus Publica's production deployment, the decision log is streamed to the central log store, where it is queryable by the on-call engineer. The decision log is the primary debugging tool: when a policy produces a surprising result in production, the engineer retrieves the decision by its decision ID (correlated via the OpenTelemetry trace ID) and replays the evaluation locally with the recorded input. The decision log is also the audit trail: every policy decision is reconstructible from the log, which is a requirement of the auditability pillar of certification.

```
#  bash  —  querying  the  decision  log  for  a  specific  decision  ID
opa                                decision-logs                        query                                \
                                   --server                        https://opa.opuspublica.svc          \
                                   --decision-id                    5f4dcc3b-5aa8-4d8c-9b8e-7e2a9c1b0001
```

9.2 The Policy Bundle

Policies are distributed to OPA instances (CI runners, production sidecars, Gatekeeper) via the OPA bundle mechanism. A bundle is a tarball of the policies/ directory, plus a .manifest file that declares the bundle's revision (the git SHA of the commit at which the bundle was built). The bundle is built by CI on every merge to main, signed with Cosign, and published to the bundle server (a simple HTTP server that OPA polls). Every OPA instance polls the bundle server at a configurable interval (default 30 seconds), downloads the latest bundle, verifies its signature, and activates it. The activation is atomic: an OPA instance either fully switches to the new bundle or continues using the old one. The bundle's revision is recorded in every decision the OPA instance makes, which means the decision log includes the policy version that produced the decision. This is the foundation of policy reproducibility: any decision can be replayed against the exact policy version that produced it.

9.3 Debugging a False Positive

A false positive is a policy that fires when it should not: the policy denies a valid input. The debugging procedure has four steps. First, retrieve the decision from the OPA decision log by its decision ID. Second, save the decision's input to a local file (input.json). Third, run the policy locally against the saved input: `opa eval --data policies/ --input input.json --format pretty 'data.opus_publica.invariants.deny'`. Fourth, inspect the policy's evaluation step-by-step using the OPA REPL (`opa eval --data policies/ --input input.json --explain fails 'data.opus_publica.invariants.deny'`), which prints, for each rule that did not fire, the expression that failed. The false positive is the rule whose body satisfied when it should not have; the

explanation shows which expression(s) the engineer mis-specified. The fix is to revise the rule, add a regression test that covers the false-positive input, and open a pull request.

```
# bash — debugging a false positive with --explain fails
opa eval \
  --data policies/ \
  --input input.json \
  --format pretty \
  --explain fails \
  'data.opus_publica.invariants.deny'

# The output shows, for each rule that did NOT fire, the expression that
# failed. For a false positive, look for a rule that DID fire and inspect
# its body to find the mis-specified expression.
```

9.4 Debugging a False Negative

A false negative is a policy that does not fire when it should: the policy admits an invalid input. False negatives are harder to detect than false positives because they produce no decision-log entry; the policy simply returns no deny messages. The detection mechanism is the parity dashboard, which compares the policy's decision against a known-good reference (the runtime monitor that the Constitution requires). When the parity dashboard shows a divergence — the runtime monitor reports a violation that the policy did not — the engineer investigates. The investigation procedure is the reverse of the false-positive procedure: retrieve the runtime monitor's evidence, construct the input that the policy SHOULD have denied, run the policy locally against the constructed input, and use `--explain fails` to identify which rule's body did not satisfy. The fix is to add or revise a rule so the body satisfies on the constructed input; add a regression test; and open a pull request.

9.5 Policy Performance

Rego evaluation has a cost, and a policy that is correct but slow can degrade CI performance or, in the production sidecar case, add latency to requests. OPA ships with a profiler (`opa eval --profile`) that reports, for each rule, the number of times it was evaluated and the time it consumed. The profiler is the engineer's primary tool for diagnosing slow policies. The common causes of slow Rego are: nested comprehensions over large inputs (the comprehension is re-evaluated for each outer iteration), the absence of an index on a frequently-queried field, and the use of `walk(input)` where a more targeted traversal would suffice. The fixes are, respectively: hoist the comprehension into a helper rule (which OPA memoizes), restructure the input so the field is a key (which OPA indexes), and replace `walk` with a direct reference. The Opus Publica performance budget for a policy is 10 ms per evaluation on a representative input; a policy that exceeds the budget is a defect and SHALL be optimized before it is merged.

```

#          bash          —          profiling          a          policy
opa          eval          \
          --data          policies/          \
          --input          evidence/INV-I-02/sample-pass.json          \
          --format          pretty          \
          --profile          \
          'data.opus_publica.invariants.deny'

#          The          output          shows,          per          rule:
#          -          the          number          of          times          the          rule          was          evaluated
#          -          the          cumulative          time          spent          in          the          rule
#          -          the          expression          that          consumed          the          most          time
#          A          rule          whose          evaluation          count          is          high          relative          to          the          input          size          is          a
#          candidate          for          hoisting          into          a          memoized          helper.

```

Chapter 10 — Counterarguments and Limitations

10.1 Limitation: Rego's Learning Curve

Rego is a Datalog-derived language, and Datalog is unfamiliar to most engineers whose background is in imperative or functional languages. The closed-world assumption, negation-as-failure, and the rule-based structure are concepts that take time to internalize. The learning curve is real, and it is a cost the team pays once per engineer. The mitigation is the OPA interactive tutorial, the patterns catalogue in Chapter 5 of this Guide, and the team's practice of pairing an experienced Rego author with a newcomer on the newcomer's first three policies. After three policies, the patterns are familiar; after ten, they are automatic. The learning curve is a one-time cost; the benefits of declarative policy (composability, testability, auditability) recur for the life of every policy.

10.2 Limitation: Gaming the Letter While Violating the Spirit

A Rego policy encodes a provision's predicate, not its intent. A policy that asserts `COUNT(dois) = 1` admits an input with one DOI that is wrong (a DOI registered for a different article), stale (a DOI that has been deleted from Crossref), or misleading (a DOI that resolves to a placeholder). The policy is correct as far as the predicate goes, but the predicate is not the whole provision. The mitigation is the conjunctive structure of Opus Publica's predicates: INV-I-02's predicate has two conjuncts (`count = 1 AND resolvable`), and a policy that encodes both is harder to game than one that encodes only the first. The further mitigation is the human reviewer, who reads the policy against the provision and asks, for each deny rule, "does this rule capture what the provision means, or only what it says?" The reviewer's judgement is the ultimate defense against letter-versus-spirit gaming; no policy alone can provide it.

10.3 Limitation: Policy Evaluation Adds Latency

Every policy evaluation takes time. In CI, the time is the wall-clock duration of the `opa eval` or `conftest` test command; for a typical Opus Publica policy, this is under 100 ms, and the cost is acceptable. In the production sidecar, the time is added to every request the sidecar evaluates; for a typical policy, this is under 1 ms, but it accumulates across the policy suite (a request that triggers ten policies adds 10 ms). In the Gatekeeper admission webhook, the time is added to every Kubernetes API call, which can slow deployments. The mitigations are the performance budget (Chapter 9), the profiler, and the engineering practice of optimizing policies that exceed the budget. The residual cost is the price of policy-as-code; the team has judged that the cost is justified by the benefit (provisions enforced continuously, without human intervention).

10.4 Counterargument: "Why Not Just Use if-Statements in Code?"

A critic might argue that the team could enforce the same provisions with if-statements in the application code: every place the code creates a DOI, add a check that the count is 1. The argument has

surface plausibility but fails on three counts. First, the if-statements are scattered: the same provision is enforced in N places, where N is the number of code paths that touch the relevant data; a change to the provision requires finding and updating all N places, which is error-prone. Second, the if-statements are untestable as a unit: there is no way to verify that the provision is enforced without exercising every code path, which is impractical. Third, the if-statements are unauditable: an external auditor cannot, by inspecting the code, verify that the provision is enforced everywhere; the auditor must trust the team's self-attestation. Rego centralizes the policy (one place per provision), makes it testable as a unit (one test file per provision), and makes it auditable (the policy is the authoritative statement of what is enforced). The if-statement approach is cheaper on the first implementation; the Rego approach is cheaper over the life of the provision.

10.5 Counterargument: “This Is Over-Engineering”

A critic might argue that the policy-as-code stack — OPA, Rego, Conftest, Gatekeeper, OSCAL, the bundle server, the decision log — is over-engineering for a platform of Opus Publica's scale. The argument fails on the scale of the provisions: 600+ certification criteria, 38 invariants, 47 non-negotiable rules, each of which MUST be enforced continuously. The cost of human enforcement of 600+ criteria is the cost of an RCA who can evaluate, by hand, every criterion per release; this cost is not sustainable, and the RC1 post-mortem identified it as the principal limitation of the RC1 model. The cost of the policy-as-code stack is the cost of building and operating the stack; this cost is paid once and amortized over every release. The break-even point, for Opus Publica's provision count, is the second release. After the second release, the policy-as-code stack is cheaper than human enforcement, and the gap widens with every release. The criticism would be valid for a platform with ten provisions; it is not valid for a platform with six hundred.

Chapter 11 — Conclusion and Implications

11.1 Summary

This Guide has taught the engineer and the CI/CD integrator how to write, test, deploy, and operate Rego policies for the Opus Publica Engineering Constitution. The stack is OPA (the engine), Rego (the language), Conftest (the pre-merge runner), GitHub Actions (the CI/CD substrate), and Gatekeeper (the deploy-time admission controller). The lifecycle is author, test, review, merge, deploy, monitor. The patterns are uniqueness, immutability, resolvability, completeness, legality, auditability, idempotency, and threshold enforcement. The testing regime is opa test for logic, conftest test for integration, meta-policies for coverage and message format. The CI/CD integration is four workflows (policy-test, conftest-verify, nightly-criterion, release-certification) emitting OSCAL-AR fragments to the certification repository. The operational tools are the decision log, the bundle, the profiler, and the parity dashboard. The counterarguments have been answered; the limitations have been acknowledged.

11.2 Implications for the Engineer's Daily Workflow

The engineer who has internalized this Guide will find their daily workflow changed in three respects. First, the engineer will read the Constitution differently: every provision is now a potential Rego policy, and the engineer will mentally translate provisions into predicates and predicates into deny rules. Second, the engineer will write tests differently: every policy is now born with a test suite, and the test suite is part of the pull request, not an afterthought. Third, the engineer will debug differently: every policy decision is reconstructible from the decision log, and the engineer will reach for the decision ID and the OPA REPL before reaching for the application's logs. The cumulative effect is a workflow in which the Constitution is not a document the engineer reads but a system the engineer operates.

11.3 Future Evolution

Two evolutionary paths are visible from the vantage of this Guide. The first is AI-assisted policy authoring: a large language model that drafts a Rego policy from a natural-language provision statement, leaving the engineer to review and test the draft. The patterns catalogue in Chapter 5 is the training data for such an assistant, and the test regime in Chapter 6 is the safety net. The second is predictive compliance: a system that, given a proposed code change, predicts which policies the change will violate, before the engineer runs the CI. The prediction is feasible because the policies are declarative and their inputs are well-typed; the prediction is useful because it shortens the feedback loop from minutes (CI) to seconds (IDE). Both paths are speculative; both are consistent with the architecture this Guide specifies; both are likely to be the subject of RC3 work. The engineer who masters this Guide is the engineer who will build those future systems.

Appendices

Appendix A — Rego Quick Reference

This appendix is a quick reference for the Rego syntax and built-in functions most used in Opus Publica policies. It is not a substitute for the canonical Rego reference at openpolicyagent.org/docs/latest/policy-reference/; it is a memory aid for the engineer who has already read the canonical reference.

A.1 Syntax

#	Package	declaration	(first	non-comment	line)
package				opus_publica.invariants	
#				Imports	
import				future.keywords.in	
import				future.keywords.every	
import				data.opus_publica.constitution.invariants.INV_I_02	
#	Rule	(set	rule,	partial)	
deny[msg]				{	
	some	v	in	input.published_versions	
		v.state	==	"published"	
	count([d d := input.dois[_]; d.article_version_id == v.id]) != 1				
	msg := sprintf("INV-I-02 FAIL: ...", [v.id])				
}					
#	Rule	(boolean,		complete)	
has_doi(version_id)				{	
	some	d	in	input.dois	
	d.article_version_id	==		version_id	
}					
#				Negation	
not				has_doi(v.id)	
#	Universal	quantification		(every)	
every	v	in	input.published_versions	{	
	v.state	==	"published"	implies	has_doi(v.id)
}					

A.2 Common Built-ins

Built-in	Signature	Returns
count	count(x)	Number of elements in array/set/object.

concat	concat(sep, xs)	xs joined by sep.
sprintf	sprintf(fmt, args)	fmt with args substituted.
regex.match	regex.match(re, s)	true if s matches re.
regex.find_n	regex.find_n(re, s, n)	First n matches of re in s.
time.now_ns	time.now_ns()	Wall-clock time in ns since epoch.
time.parse_rfc3339_ns	time.parse_rfc3339_ns(s)	Parse RFC 3339 string to ns.
object.get	object.get(o, k, default)	o[k] or default if k absent.
object.remove	object.remove(o, keys)	o without the keys in the set.
lower / upper	lower(s) / upper(s)	Case conversion.
contains	contains(s, sub)	true if s contains sub.
startswith / endswith	startswith(s, prefix)	true if s starts with prefix.
split	split(s, sep)	s split by sep into array.
trim	trim(s, cutset)	s with leading/trailing cutset chars removed.
to_number	to_number(s)	Number parsed from string s.
walk	walk(x)	Array of [path, value] pairs for every node in x.
sum / max / min	sum(xs) / max(xs) / min(xs)	Aggregate of array.
sort	sort(xs)	Sorted copy of array.
array.concat	array.concat(a, b)	Concatenation of two arrays.
object.filter	object.filter(o, keys)	o restricted to keys.

Appendix B — Opus Publica Rego Style Guide

This appendix specifies the style conventions every Opus Publica Rego policy SHALL follow. The conventions are enforced by the meta-policies in policies/_meta/ and by the reviewer's judgement. The conventions exist to make policies uniform across the team, so that any engineer can read any policy and understand it without learning a new style.

Convention	Specification
File naming	policies/<PROVISION-ID>.rego. One file per provision. Provision ID in UPPER-CASE with hyphens (e.g., INV-I-02.rego).
Test file naming	tests/<PROVISION-ID>_test.rego. The _test suffix is


```

#      import      data.opus_publica.constitution.<path>      as      needed

#  Deny  rule  1:  <one-sentence statement of what this rule catches>
deny[msg]
      some      <binding>      in      input.<collection>
                <predicate      expression>
                msg      :=      sprintf(
      "<PROVISION-ID>  FAIL:  <human-readable explanation including %s>",
                                [<binding>.<id>]
                                )
}

#      Helper      rule:      <what      it      computes>
<helper_name>(<arg>)
      some      <binding>      in      input.<collection>
                <predicate      expression>
}

```

The corresponding test template is:

```

#      tests/<PROVISION-ID>_test.rego
package      opus_publica.<class>

import      future.keywords.in

#      PASS      case:      <what      is      being      tested>
test_pass_<scenario>
      count(deny)      ==      0      with      input      as      {
                                <inline      input>
                                }
}

#      FAIL      case:      <what      is      being      tested>
test_fail_<scenario>
      some      msg      in      deny      with      input      as      {
                                <inline      input>
                                }
      contains(msg,      "<PROVISION-ID>      FAIL")
}

```

Appendix D — References

The references below are the canonical sources for the technologies and standards this Guide cites. The engineer is expected to consult them for authoritative detail. URLs are current as of the issue date of this Guide.

Reference	URL	Authority
Open Policy Agent (homepage)	https://www.openpolicyagent.org	CNCF Graduated project; canonical OPA reference.
OPA documentation	https://www.openpolicyagent.org/docs/latest/	Canonical OPA documentation.
Rego policy reference	https://www.openpolicyagent.org/docs/latest/policy-reference/	Rego language specification and built-in catalog.
OPA tutorial	https://www.openpolicyagent.org/docs/latest/#tutorials	Interactive OPA tutorial; recommended starting point.
Conftest (homepage)	https://www.conftest.dev/	OPA subproject; canonical Conftest reference.
Conftest GitHub	https://github.com/open-policy-agent/conftest	Conftest source repository and releases.
OPA Gatekeeper	https://open-policy-agent.github.io/gatekeeper/website/	Canonical Gatekeeper (Kubernetes admission controller) reference.
Gatekeeper constraint templates library	https://github.com/open-policy-agent/gatekeeper-library	Community library of Gatekeeper constraint templates.
NIST SP 800-204C	https://csrc.nist.gov/pubs/sp/800/204/c/final	Implementation of DevSecOps for a Microservices-based Application with Service Mesh; policy-as-code guidance.
NIST SP 800-137	https://csrc.nist.gov/pubs/sp/800/137/final	Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations.
GitHub Actions documentation	https://docs.github.com/en/actions	Canonical GitHub Actions reference.
OSCAL (Open Security Controls Assessment Language)	https://pages.nist.gov/OSCAL/	NIST-led machine-readable format for security control documentation.
OSCAL Assessment Results model	https://pages.nist.gov/OSCAL/concepts/layer/assessment/assessment-results/	The OSCAL layer the CCE emits.
RFC 2119	https://www.rfc-editor.org/rfc/	Key words for use in RFCs to

	rfc2119	indicate requirement levels.
IETF RFC 6902 (JSON Patch)	https://www.rfc-editor.org/rfc/rfc6902	Used by constitutional diffing (out of scope for this Guide).
SLSA (Supply-chain Levels for Software Artifacts)	https://slsa.dev/	Supply-chain provenance framework; the CCE targets SLSA L3.
Sigstore (Cosign, Rekor, Fulcio)	https://www.sigstore.dev/	Signing, transparency log, and CA for software artifacts.
CNCF Graduated projects list	https://www.cncf.io/projects/	Confirms OPA's CNCF Graduated status (February 2021).